SQL Notes

Quick-reference guide for interview preparation — clauses, transactions, grouping & joins

---

1. Basic Querying

1. SELECT

Used to retrieve data from a table.

```
SELECT *
FROM Customers;
```

2. DISTINCT

Used to get unique values by removing duplicates.

```
SELECT DISTINCT City
FROM Customers;
```

3. WHERE

Used to filter rows based on a condition.

```
SELECT *
FROM Customers
WHERE Age > 20;
```

Multiple conditions:

```
SELECT *
FROM Customers
WHERE City = 'Kakinada' AND Age > 20;
```

4. LIKE (Pattern Matching)

Used for pattern matching within text columns.

Starts with 'A':

```
SELECT *
FROM Customers
WHERE Name LIKE 'A%';
```

Ends with 'a':

```
SELECT *
FROM Customers
WHERE Name LIKE '%a';
```

Contains 'an':

```
SELECT *
FROM Customers
WHERE Name LIKE '%an%';
```

Exactly 5 characters:

```
SELECT *
FROM Customers
WHERE Name LIKE '_____';
```

Remember: % = any number of characters (including zero). _ = exactly one character.

5. ORDER BY

Used to sort the result set.

Ascending (default):

```
SELECT *
FROM Customers
ORDER BY Age ASC;
```

Descending:

```
SELECT *
FROM Customers
ORDER BY Age DESC;
```

Remember: ASC = smallest → largest. DESC = largest → smallest.

6. TOP

Used to limit the number of rows returned (SQL Server syntax).

```
SELECT TOP 5 *
FROM Customers;
```

Top 5 oldest customers:

```
SELECT TOP 5 *
FROM Customers
ORDER BY Age DESC;
```

TOP with percentage:

```
SELECT TOP 10 PERCENT *
FROM Customers;
```

7. RENAME COLUMN

In SQL Server, columns are renamed using the sp_rename system procedure.

```
EXEC sp_rename 'Customers.Phone', 'MobileNumber', 'COLUMN';
```

2. Transaction Control (TCL)

8. COMMIT

COMMIT permanently saves all changes made in the current transaction.

```
BEGIN TRANSACTION;

UPDATE Customers
SET City = 'Hyderabad'
WHERE CustomerID = 1;

COMMIT;
```

After COMMIT, the change is saved permanently and cannot be rolled back.

Remember: COMMIT = Save permanently.

9. ROLLBACK

ROLLBACK cancels/undoes changes made inside the current (uncommitted) transaction.

```
BEGIN TRANSACTION;

UPDATE Customers
SET City = 'Chennai'
WHERE CustomerID = 1;

ROLLBACK;
```

The update above is cancelled — the City value reverts to what it was before.

Remember: ROLLBACK = Undo.

10. SAVEPOINT

A savepoint marks a point within a transaction that you can roll back to, without undoing the entire transaction. In SQL Server, this is done with SAVE TRANSACTION.

```
BEGIN TRANSACTION;

UPDATE Customers
SET City = 'Hyderabad'
WHERE CustomerID = 1;

SAVE TRANSACTION S1;

UPDATE Customers
SET City = 'Chennai'
WHERE CustomerID = 2;

ROLLBACK TRANSACTION S1;

COMMIT;
```

Here, the second update (CustomerID = 2) is rolled back to savepoint S1, but the first update (CustomerID = 1) is still committed.

3. Filtering, Ranges & Date Functions

11. DATEDIFF

Used to calculate the difference between two dates.

Syntax:

```
DATEDIFF(datepart, startdate, enddate)
```

Difference in days:

```
SELECT DATEDIFF(DAY, '2026-01-01', '2026-01-10') AS Days;
```

12. BETWEEN

Used to select values within a range (inclusive of both endpoints).

```
SELECT *
FROM Customers
WHERE Age BETWEEN 20 AND 30;
```

This includes both 20 and 30. Equivalent to:

```
WHERE Age >= 20 AND Age <= 30
```

Works with dates too:

```
SELECT *
FROM Customers
WHERE OrderDate BETWEEN '2026-01-01' AND '2026-01-31';
```

13. NOT IN

Used to exclude multiple specific values.

```
SELECT *
FROM Customers
WHERE City NOT IN ('Kakinada', 'Hyderabad');
```

This returns customers whose city is neither Kakinada nor Hyderabad.

Clause	Meaning
WHERE City IN ('Kakinada','Hyderabad')	Only these cities
WHERE City NOT IN ('Kakinada','Hyderabad')	Exclude these cities

4. Grouping & Aggregation

14. GROUP BY

GROUP BY groups rows that have the same values in specified columns, typically used with aggregate functions like COUNT, SUM, AVG, MIN, MAX.

```
SELECT City, COUNT(*) AS CustomerCount
FROM Customers
GROUP BY City;
```

15. HAVING

HAVING filters groups created by GROUP BY (WHERE cannot filter on aggregate results).

Show only cities having more than 1 customer:

```
SELECT
City,
COUNT(*) AS CustomerCount
FROM Customers
GROUP BY City
HAVING COUNT(*) > 1;
```

16. GROUP BY + WHERE + HAVING together

WHERE filters rows *before* grouping; HAVING filters groups *after* aggregation.

```
SELECT
City,
COUNT(*) AS CustomerCount
FROM Customers
WHERE Age >= 20
GROUP BY City
HAVING COUNT(*) > 1;
```

Remember: Order of execution: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

5. SQL Joins (6 Types)

Joins combine rows from two or more tables based on a related column. Assume two sample tables: **Customers**(CustomerID, Name, City) and **Orders**(OrderID, CustomerID, Amount).

17. INNER JOIN

Returns only the rows that have matching values in both tables.

```
SELECT c.Name, o.OrderID, o.Amount
FROM Customers c
INNER JOIN Orders o
ON c.CustomerID = o.CustomerID;
```

Remember: INNER JOIN = only common/matching records from both tables.

18. LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the left table, and matching rows from the right table. Unmatched right-side columns return NULL.

```
SELECT c.Name, o.OrderID, o.Amount
FROM Customers c
LEFT JOIN Orders o
ON c.CustomerID = o.CustomerID;
```

Remember: LEFT JOIN = all of the left table + matches from the right (NULLs where no match).

19. RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table, and matching rows from the left table. Unmatched left-side columns return NULL.

```
SELECT c.Name, o.OrderID, o.Amount
FROM Customers c
RIGHT JOIN Orders o
ON c.CustomerID = o.CustomerID;
```

Remember: RIGHT JOIN = all of the right table + matches from the left.

20. FULL JOIN (FULL OUTER JOIN)

Returns all rows from both tables, matching where possible. Non-matching rows on either side are filled with NULL.

```
SELECT c.Name, o.OrderID, o.Amount
FROM Customers c
FULL OUTER JOIN Orders o
ON c.CustomerID = o.CustomerID;
```

Remember: FULL JOIN = LEFT JOIN + RIGHT JOIN combined (everything from both sides).

21. CROSS JOIN

Returns the Cartesian product — every row of the first table combined with every row of the second table. No ON condition is used.

```
SELECT c.Name, o.OrderID
FROM Customers c
CROSS JOIN Orders o;
```

Remember: CROSS JOIN = no matching condition; result rows = (rows in table1) x (rows in table2).

22. SELF JOIN

A table is joined with itself, typically to compare rows within the same table (e.g., finding employees and their managers). Requires table aliases.

```
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employees e
INNER JOIN Employees m
ON e.ManagerID = m.EmployeeID;
```

Remember: SELF JOIN = same table used twice with different aliases.

. Join Types — Quick Comparison

Join Type	Returns
INNER JOIN	Only matching rows in both tables
LEFT JOIN	All left rows + matches from right (NULL if none)
RIGHT JOIN	All right rows + matches from left (NULL if none)
FULL OUTER JOIN	All rows from both tables (NULL where no match)
CROSS JOIN	Cartesian product — all combinations
SELF JOIN	Table joined with itself using aliases

6. Quick Interview Tips

• **WHERE** filters rows before grouping; **HAVING** filters groups after aggregation.

• **DISTINCT** removes duplicate rows from the result, not from the table itself.

• **COMMIT** is permanent; **ROLLBACK** only works before COMMIT is executed.

• **LEFT/RIGHT JOIN** direction refers to which table keeps ALL its rows.

• A query with no ON/matching condition between two tables behaves like a **CROSS JOIN**.

• Logical order of query execution: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → TOP/LIMIT.